Chapter 5. Querying Multiple Tables

Back in Chapter 2, I demonstrated how related concepts are broken into separate pieces through a process known as normalization. The end result of this exercise was two tables: `person` and `favorite_food`. If, however, you want to generate a single report showing a person's name, address, *and* favorite foods, you will need a mechanism to bring the data from these two tables back together again; this mechanism is known as a *join*, and this chapter concentrates on the simplest and most common join, the *inner join*. Chapter 10 demonstrates all of the different join types.

What Is a Join?

Queries against a single table are certainly not rare, but you will find that most of your queries will require two, three, or even more tables. To illustrate, let's look at the definitions for the `customer` and `address` tables and then define a query that retrieves data from both tables:

```
mysql> desc customer;
+-------------+----------------------+------+-----+-------------------+
| Field       | Type                 | Null | Key | Default           |
+-------------+----------------------+------+-----+-------------------+
| customer_id | smallint(5) unsigned | NO   | PRI | NULL              |
| store_id    | tinyint(3) unsigned  | NO   | MUL | NULL              |
| first_name  | varchar(45)          | NO   |     | NULL              |
| last_name   | varchar(45)          | NO   | MUL | NULL              |
| email       | varchar(50)          | YES  |     | NULL              |
| address_id  | smallint(5) unsigned | NO   | MUL | NULL              |
| active      | tinyint(1)           | NO   |     | 1                 |
| create_date | datetime             | NO   |     | NULL              |
| last_update | timestamp            | YES  |     | CURRENT_TIMESTAMP |
+-------------+----------------------+------+-----+-------------------+

mysql> desc address;
+-------------+----------------------+------+-----+-------------------+
| Field       | Type                 | Null | Key | Default           |
+-------------+----------------------+------+-----+-------------------+
```

```
| address_id  | smallint(5) unsigned | NO   | PRI | NULL              |
| address     | varchar(50)          | NO   |     | NULL              |
| address2    | varchar(50)          | YES  |     | NULL              |
| district    | varchar(20)          | NO   |     | NULL              |
| city_id     | smallint(5) unsigned | NO   | MUL | NULL              |
| postal_code | varchar(10)          | YES  |     | NULL              |
| phone       | varchar(20)          | NO   |     | NULL              |
| location    | geometry             | NO   | MUL | NULL              |
| last_update | timestamp            | NO   |     | CURRENT_TIMESTAMP |
+-------------+----------------------+------+-----+-------------------+
```

Let's say you want to retrieve the first and last names of each customer, along with their street address. Your query will therefore need to retrieve the `customer.first_name`, `customer.last_name`, and `address.address` columns. But how can you retrieve data from both tables in the same query? The answer lies in the `customer.address_id` column, which holds the ID of the customer's record in the `address` table (in more formal terms, the `customer.address_id` column is the *foreign key* to the `address` table). The query, which you will see shortly, instructs the server to use the `customer.address_id` column as the *transportation* between the `customer` and `address` tables, thereby allowing columns from both tables to be included in the query's result set. This type of operation is known as a *join*.

NOTE

A foreign key constraint can optionally be created to verify that the values in one table exist in another table. For the previous example, a foreign key constraint could be created on the `customer` table to ensure that any values inserted into the `customer.address_id` column can be found in the `address.address_id` column. Please note that it is not necessary to have a foreign key constraint in place in order to join two tables.

Cartesian Product

The easiest way to start is to put the `customer` and `address` tables into the `from` clause of a query and see what happens. Here's a query that retrieves the customer's first and last names along with the street address, with a `from` clause naming both tables separated by the `join` keyword:

```
mysql> SELECT c.first_name, c.last_name, a.address
    -> FROM customer c JOIN address a;
+------------+-----------+----------------------+
```

```
| first_name | last_name | address             |
+------------+-----------+---------------------+
| MARY       | SMITH     | 47 MySakila Drive   |
| PATRICIA   | JOHNSON   | 47 MySakila Drive   |
| LINDA      | WILLIAMS  | 47 MySakila Drive   |
| BARBARA    | JONES     | 47 MySakila Drive   |
| ELIZABETH  | BROWN     | 47 MySakila Drive   |
| JENNIFER   | DAVIS     | 47 MySakila Drive   |
| MARIA      | MILLER    | 47 MySakila Drive   |
| SUSAN      | WILSON    | 47 MySakila Drive   |
...
| SETH       | HANNON    | 1325 Fukuyama Street |
| KENT       | ARSENAULT | 1325 Fukuyama Street |
| TERRANCE   | ROUSH     | 1325 Fukuyama Street |
| RENE       | MCALISTER | 1325 Fukuyama Street |
| EDUARDO    | HIATT     | 1325 Fukuyama Street |
| TERRENCE   | GUNDERSON | 1325 Fukuyama Street |
| ENRIQUE    | FORSYTHE  | 1325 Fukuyama Street |
| FREDDIE    | DUGGAN    | 1325 Fukuyama Street |
| WADE       | DELVALLE  | 1325 Fukuyama Street |
| AUSTIN     | CINTRON   | 1325 Fukuyama Street |
+------------+-----------+---------------------+
361197 rows in set (0.03 sec)
```

Hmmm...there are only 599 customers and 603 rows in the `address` table, so how did the result set end up with 361,197 rows? Looking more closely, you can see that many of the customers seem to have the same street address. Because the query didn't specify *how* the two tables should be joined, the database server generated the *Cartesian product*, which is *every* permutation of the two tables (599 customers x 603 addresses = 361,197 permutations). This type of join is known as a *cross join*, and it is rarely used (on purpose, at least). Cross joins are one of the join types that we study in Chapter 10.

Inner Joins

To modify the previous query so that only a single row is returned for each customer, you need to describe how the two tables are related. Earlier, I showed that the `customer.address_id` column serves as the link between the two tables, so this information needs to be added to the `on` subclause of the `from` clause:

```
mysql> SELECT c.first_name, c.last_name, a.address
    -> FROM customer c JOIN address a
    ->   ON c.address_id = a.address_id;
```

```
+-------------+--------------+-------------------------------------+
| first_name  | last_name    | address                             |
+-------------+--------------+-------------------------------------+
| MARY        | SMITH        | 1913 Hanoi Way                      |
| PATRICIA    | JOHNSON      | 1121 Loja Avenue                    |
| LINDA       | WILLIAMS     | 692 Joliet Street                   |
| BARBARA     | JONES        | 1566 Inegl Manor                    |
| ELIZABETH   | BROWN        | 53 Idfu Parkway                     |
| JENNIFER    | DAVIS        | 1795 Santiago de Compostela Way     |
| MARIA       | MILLER       | 900 Santiago de Compostela Parkway  |
| SUSAN       | WILSON       | 478 Joliet Way                      |
| MARGARET    | MOORE        | 613 Korolev Drive                   |
...
| TERRANCE    | ROUSH        | 42 Fontana Avenue                   |
| RENE        | MCALISTER    | 1895 Zhezqazghan Drive              |
| EDUARDO     | HIATT        | 1837 Kaduna Parkway                 |
| TERRENCE    | GUNDERSON    | 844 Bucuresti Place                 |
| ENRIQUE     | FORSYTHE     | 1101 Bucuresti Boulevard            |
| FREDDIE     | DUGGAN       | 1103 Quilmes Boulevard              |
| WADE        | DELVALLE     | 1331 Usak Boulevard                 |
| AUSTIN      | CINTRON      | 1325 Fukuyama Street                |
+-------------+--------------+-------------------------------------+
599 rows in set (0.00 sec)
```

Instead of 361,197 rows, you now have the expected 599 rows due to the addition of the `on` subclause, which instructs the server to join the `customer` and `address` tables by using the `address_id` column to traverse from one table to the other. For example, Mary Smith's row in the `customer` table contains a value of `5` in the `address_id` column (not shown in the example). The server uses this value to look up the row in the `address` table having a value of `5` in its `address_id` column and then retrieves the value `'1913 Hanoi Way'` from the `address` column in that row.

If a value exists for the `address_id` column in one table but *not* the other, then the join fails for the rows containing that value, and those rows are excluded from the result set. This type of join is known as an *inner join*, and it is the most commonly used type of join. To clarify, if a row in the `customer` table has the value 999 in the `address_id` column and there's no row in the `address` table with a value of 999 in the `address_id` column, then that customer row would not be included in the result set. If you want to include all rows from one table or the other regardless of whether a match exists, you need to specify an *outer join*, but this will be explored Chapter 10.

In the previous example, I did not specify in the `from` clause which type of join to use. However, when you wish to join two tables using an inner join, you should explicitly

specify this in your `from` clause; here's the same example, with the addition of the join type (note the keyword `inner`):

```
SELECT c.first_name, c.last_name, a.address
FROM customer c INNER JOIN address a
  ON c.address_id = a.address_id;
```

If you do not specify the type of join, then the server will do an inner join by default. As you will see later in the book, however, there are several types of joins, so you should get in the habit of specifying the exact type of join that you require, especially for the benefit of any other people who might use/maintain your queries in the future.

If the names of the columns used to join the two tables are identical, which is true in the previous query, you can use the `using` subclause instead of the `on` subclause, as in:

```
SELECT c.first_name, c.last_name, a.address
FROM customer c INNER JOIN address a
  USING (address_id);
```

Since `using` is a shorthand notation that you can use in only a specific situation, I prefer always to use the `on` subclause to avoid confusion.

The ANSI Join Syntax

The notation used throughout this book for joining tables was introduced in the SQL92 version of the ANSI SQL standard. All the major databases (Oracle Database, Microsoft SQL Server, MySQL, IBM DB2 Universal Database, and Sybase Adaptive Server) have adopted the SQL92 join syntax. Because most of these servers have been around since before the release of the SQL92 specification, they all include an older join syntax as well. For example, all these servers would understand the following variation of the previous query:

```
mysql> SELECT c.first_name, c.last_name, a.address
    -> FROM customer c, address a
    -> WHERE c.address_id = a.address_id;
+------------+------------+-----------------------------------+
| first_name | last_name  | address                           |
+------------+------------+-----------------------------------+
| MARY       | SMITH      | 1913 Hanoi Way                    |
| PATRICIA   | JOHNSON    | 1121 Loja Avenue                  |
```

```
| LINDA     | WILLIAMS   | 692 Joliet Street                |
| BARBARA   | JONES      | 1566 Inegl Manor                 |
| ELIZABETH | BROWN      | 53 Idfu Parkway                  |
| JENNIFER  | DAVIS      | 1795 Santiago de Compostela Way  |
| MARIA     | MILLER     | 900 Santiago de Compostela Parkway |
| SUSAN     | WILSON     | 478 Joliet Way                   |
| MARGARET  | MOORE      | 613 Korolev Drive                |
...
| TERRANCE  | ROUSH      | 42 Fontana Avenue                |
| RENE      | MCALISTER  | 1895 Zhezqazghan Drive           |
| EDUARDO   | HIATT      | 1837 Kaduna Parkway              |
| TERRENCE  | GUNDERSON  | 844 Bucuresti Place              |
| ENRIQUE   | FORSYTHE   | 1101 Bucuresti Boulevard         |
| FREDDIE   | DUGGAN     | 1103 Quilmes Boulevard           |
| WADE      | DELVALLE   | 1331 Usak Boulevard              |
| AUSTIN    | CINTRON    | 1325 Fukuyama Street             |
+-----------+------------+----------------------------------+
599 rows in set (0.00 sec)
```

This older method of specifying joins does not include the `on` subclause; instead, tables are named in the `from` clause separated by commas, and join conditions are included in the `where` clause. While you may decide to ignore the SQL92 syntax in favor of the older join syntax, the ANSI join syntax has the following advantages:

- Join conditions and filter conditions are separated into two different clauses (the `on` subclause and the `where` clause, respectively), making a query easier to understand.
- The join conditions for each pair of tables are contained in their own `on` clause, making it less likely that part of a join will be mistakenly omitted.
- Queries that use the SQL92 join syntax are portable across database servers, whereas the older syntax is slightly different across the different servers.

The benefits of the SQL92 join syntax are easier to identify for complex queries that include both join and filter conditions. Consider the following query, which returns only those customers whose postal code is 52137:

```
mysql> SELECT c.first_name, c.last_name, a.address
    -> FROM customer c, address a
    -> WHERE c.address_id = a.address_id
    ->   AND a.postal_code = 52137;
+------------+-----------+----------------------+
| first_name | last_name | address              |
+------------+-----------+----------------------+
```

```
| JAMES     | GANNON    | 1635 Kuwana Boulevard  |
| FREDDIE   | DUGGAN    | 1103 Quilmes Boulevard |
+-----------+-----------+------------------------+
2 rows in set (0.01 sec)
```

At first glance, it is not so easy to determine which conditions in the `where` clause are join conditions and which are filter conditions. It is also not readily apparent which type of join is being employed (to identify the type of join, you would need to look closely at the join conditions in the `where` clause to see whether any special characters are employed), nor is it easy to determine whether any join conditions have been mistakenly left out. Here's the same query using the SQL92 join syntax:

```
mysql> SELECT c.first_name, c.last_name, a.address
    -> FROM customer c INNER JOIN address a
    ->   ON c.address_id = a.address_id
    -> WHERE a.postal_code = 52137;
+------------+-----------+------------------------+
| first_name | last_name | address                |
+------------+-----------+------------------------+
| JAMES      | GANNON    | 1635 Kuwana Boulevard  |
| FREDDIE    | DUGGAN    | 1103 Quilmes Boulevard |
+------------+-----------+------------------------+
2 rows in set (0.00 sec)
```

With this version, it is clear which condition is used for the join and which condition is used for filtering. Hopefully, you will agree that the version using SQL92 join syntax is easier to understand.

Joining Three or More Tables

Joining three tables is similar to joining two tables, but with one slight wrinkle. With a two-table join, there are two tables and one join type in the `from` clause, and a single `on` subclause to define how the tables are joined. With a three-table join, there are three tables and two join types in the `from` clause, and two `on` subclauses.

To illustrate, let's change the previous query to return the customer's city rather than their street address. The city name, however, is not stored in the `address` table but is accessed via a foreign key to the `city` table. Here are the table definitions:

```
mysql> desc address;
+-------------+----------------------+------+-----+-------------------+
| Field       | Type                 | Null | Key | Default           |
+-------------+----------------------+------+-----+-------------------+
| address_id  | smallint(5) unsigned | NO   | PRI | NULL              |
| address     | varchar(50)          | NO   |     | NULL              |
| address2    | varchar(50)          | YES  |     | NULL              |
| district    | varchar(20)          | NO   |     | NULL              |
| city_id     | smallint(5) unsigned | NO   | MUL | NULL              |
| postal_code | varchar(10)          | YES  |     | NULL              |
| phone       | varchar(20)          | NO   |     | NULL              |
| location    | geometry             | NO   | MUL | NULL              |
| last_update | timestamp            | NO   |     | CURRENT_TIMESTAMP |
+-------------+----------------------+------+-----+-------------------+

mysql> desc city;
+-------------+----------------------+------+-----+-------------------+
| Field       | Type                 | Null | Key | Default           |
+-------------+----------------------+------+-----+-------------------+
| city_id     | smallint(5) unsigned | NO   | PRI | NULL              |
| city        | varchar(50)          | NO   |     | NULL              |
| country_id  | smallint(5) unsigned | NO   | MUL | NULL              |
| last_update | timestamp            | NO   |     | CURRENT_TIMESTAMP |
+-------------+----------------------+------+-----+-------------------+
```

To show each customer's city, you will need to traverse from the `customer` table to the `address` table using the `address_id` column and then from the `address` table to the `city` table using the `city_id` column. The query would look like the following:

```
mysql> SELECT c.first_name, c.last_name, ct.city
    -> FROM customer c
    ->   INNER JOIN address a
    ->   ON c.address_id = a.address_id
    ->   INNER JOIN city ct
    ->   ON a.city_id = ct.city_id;
+-------------+--------------+---------------------------+
| first_name  | last_name    | city                      |
+-------------+--------------+---------------------------+
| JULIE       | SANCHEZ      | A Corua (La Corua)        |
| PEGGY       | MYERS        | Abha                      |
| TOM         | MILNER       | Abu Dhabi                 |
| GLEN        | TALBERT      | Acua                      |
| LARRY       | THRASHER     | Adana                     |
| SEAN        | DOUGLASS     | Addis Abeba               |
```

```
...
| MICHELE     | GRANT       | Yuncheng              |
| GARY        | COY         | Yuzhou                |
| PHYLLIS     | FOSTER      | Zalantun              |
| CHARLENE    | ALVAREZ     | Zanzibar              |
| FRANKLIN    | TROUTMAN    | Zaoyang               |
| FLOYD       | GANDY       | Zapopan               |
| CONSTANCE   | REID        | Zaria                 |
| JACK        | FOUST       | Zeleznogorsk          |
| BYRON       | BOX         | Zhezqazghan           |
| GUY         | BROWNLEE    | Zhoushan              |
| RONNIE      | RICKETTS    | Ziguinchor            |
+-------------+-------------+-------------------------+
599 rows in set (0.03 sec)
```

For this query, there are three tables, two join types, and two `on` subclauses in the `from` clause, so things have gotten quite a bit busier. At first glance, it might seem like the order in which the tables appear in the `from` clause is important, but if you switch the table order, you will get the exact same results. All three of these variations return the same results:

```
SELECT c.first_name, c.last_name, ct.city
FROM customer c
  INNER JOIN address a
  ON c.address_id = a.address_id
  INNER JOIN city ct
  ON a.city_id = ct.city_id;

SELECT c.first_name, c.last_name, ct.city
FROM city ct
  INNER JOIN address a
  ON a.city_id = ct.city_id
  INNER JOIN customer c
  ON c.address_id = a.address_id;

SELECT c.first_name, c.last_name, ct.city
FROM address a
  INNER JOIN city ct
  ON a.city_id = ct.city_id
  INNER JOIN customer c
  ON c.address_id = a.address_id;
```

The only difference you may see would be the order in which the rows are returned, since there is no `order by` clause to specify how the results should be ordered.

DOES JOIN ORDER MATTER?

If you are confused about why all three versions of the `customer` / `address` / `city` query yield the same results, keep in mind that SQL is a nonprocedural language, meaning that you describe what you want to retrieve and which database objects need to be involved, but it is up to the database server to determine how best to execute your query. Using statistics gathered from your database objects, the server must pick one of three tables as a starting point (the chosen table is thereafter known as the *driving table*) and then decide in which order to join the remaining tables. Therefore, the order in which tables appear in your `from` clause is not significant.

If, however, you believe that the tables in your query should always be joined in a particular order, you can place the tables in the desired order and then specify the keyword `straight_join` in MySQL, request the `force order` option in SQL Server, or use either the `ordered` or the `leading` optimizer hint in Oracle Database. For example, to tell the MySQL server to use the `city` table as the driving table and to then join the `address` and `customer` tables, you could do the following:

```
SELECT STRAIGHT_JOIN c.first_name, c.last_name, ct.city
FROM city ct
  INNER JOIN address a
  ON a.city_id = ct.city_id
  INNER JOIN customer c
  ON c.address_id = a.address_id
```

Using Subqueries as Tables

You have already seen several examples of queries that include multiple tables, but there is one variation worth mentioning: what to do if some of the data sets are generated by subqueries. Subqueries are the focus of Chapter 9, but I already introduced the concept of a subquery in the `from` clause in the previous chapter. The following query joins the `customer` table to a subquery against the `address` and `city` tables:

```
mysql> SELECT c.first_name, c.last_name, addr.address, addr.city
    -> FROM customer c
    ->   INNER JOIN
```

```
    ->    (SELECT a.address_id, a.address, ct.city
    ->     FROM address a
    ->       INNER JOIN city ct
    ->       ON a.city_id = ct.city_id
    ->     WHERE a.district = 'California'
    ->    ) addr
    ->   ON c.address_id = addr.address_id;
+------------+-----------+----------------------+---------------+
| first_name | last_name | address              | city          |
+------------+-----------+----------------------+---------------+
| PATRICIA   | JOHNSON   | 1121 Loja Avenue     | San Bernardino |
| BETTY      | WHITE     | 770 Bydgoszcz Avenue | Citrus Heights |
| ALICE      | STEWART   | 1135 Izumisano Parkway | Fontana     |
| ROSA       | REYNOLDS  | 793 Cam Ranh Avenue  | Lancaster     |
| RENEE      | LANE      | 533 al-Ayn Boulevard | Compton       |
| KRISTIN    | JOHNSTON  | 226 Brest Manor      | Sunnyvale     |
| CASSANDRA  | WALTERS   | 920 Kumbakonam Loop  | Salinas       |
| JACOB      | LANCE     | 1866 al-Qatif Avenue | El Monte      |
| RENE       | MCALISTER | 1895 Zhezqazghan Drive | Garden Grove |
+------------+-----------+----------------------+---------------+
9 rows in set (0.00 sec)
```

The subquery, which starts on line 4 and is given the alias `addr`, finds all addresses that are in California. The outer query joins the subquery results to the `customer` table to return the first name, last name, street address, and city of all customers who live in California. While this query could have been written without the use of a subquery by simply joining the three tables, it can sometimes be advantageous from a performance and/or readability aspect to use one or more subqueries.

One way to visualize what is going on is to run the subquery by itself and look at the results. Here are the results of the subquery from the prior example:

```
mysql> SELECT a.address_id, a.address, ct.city
    -> FROM address a
    ->   INNER JOIN city ct
    ->   ON a.city_id = ct.city_id
    -> WHERE a.district = 'California';
+------------+----------------------+---------------+
| address_id | address              | city          |
+------------+----------------------+---------------+
|          6 | 1121 Loja Avenue     | San Bernardino |
|         18 | 770 Bydgoszcz Avenue | Citrus Heights |
|         55 | 1135 Izumisano Parkway | Fontana     |
|        116 | 793 Cam Ranh Avenue  | Lancaster     |
```

```
|      186 | 533 al-Ayn Boulevard   | Compton       |
|      218 | 226 Brest Manor        | Sunnyvale     |
|      274 | 920 Kumbakonam Loop    | Salinas       |
|      425 | 1866 al-Qatif Avenue   | El Monte      |
|      599 | 1895 Zhezqazghan Drive | Garden Grove  |
+----------+------------------------+---------------+
9 rows in set (0.00 sec)
```

This result set consists of all nine California addresses. When joined to the `customer` table via the `address_id` column, your result set will contain information about the customers assigned to these addresses.

Using the Same Table Twice

If you are joining multiple tables, you might find that you need to join the same table more than once. In the sample database, for example, actors are related to the films in which they appeared via the `film_actor` table. If you want to find all of the films in which two specific actors appear, you could write a query such as this one, which joins the `film` table to the `film_actor` table to the `actor` table:

```
mysql> SELECT f.title
    -> FROM film f
    ->   INNER JOIN film_actor fa
    ->   ON f.film_id = fa.film_id
    ->   INNER JOIN actor a
    ->   ON fa.actor_id = a.actor_id
    -> WHERE ((a.first_name = 'CATE' AND a.last_name = 'MCQUEEN')
    ->     OR (a.first_name = 'CUBA' AND a.last_name = 'BIRCH'));
+---------------------+
| title               |
+---------------------+
| ATLANTIS CAUSE      |
| BLOOD ARGONAUTS     |
| COMMANDMENTS EXPRESS |
| DYNAMITE TARZAN     |
| EDGE KISSING        |
...
| TOWERS HURRICANE    |
| TROJAN TOMORROW     |
| VIRGIN DAISY        |
| VOLCANO TEXAS       |
| WATERSHIP FRONTIER  |
```

```
+---------------------+
54 rows in set (0.00 sec)
```

This query returns all movies in which either Cate McQueen or Cuba Birch appeared. However, let's say that you want to retrieve only those films in which *both* of these actors appeared. To accomplish this, you will need to find all rows in the `film` table that have two rows in the `film_actor` table, one of which is associated with Cate McQueen, and the other associated with Cuba Birch. Therefore, you will need to include the `film_actor` and `actor` tables twice, each with a different alias so that the server knows which one you are referring to in the various clauses:

```
mysql> SELECT f.title
    ->  FROM film f
    ->    INNER JOIN film_actor fa1
    ->    ON f.film_id = fa1.film_id
    ->    INNER JOIN actor a1
    ->    ON fa1.actor_id = a1.actor_id
    ->    INNER JOIN film_actor fa2
    ->    ON f.film_id = fa2.film_id
    ->    INNER JOIN actor a2
    ->    ON fa2.actor_id = a2.actor_id
    -> WHERE (a1.first_name = 'CATE' AND a1.last_name = 'MCQUEEN')
    ->   AND (a2.first_name = 'CUBA' AND a2.last_name = 'BIRCH');
+------------------+
| title            |
+------------------+
| BLOOD ARGONAUTS  |
| TOWERS HURRICANE |
+------------------+
2 rows in set (0.00 sec)
```

Between them, the two actors appeared in 52 different films, but there are only two films in which both actors appeared. This is one example of a query that *requires* the use of table aliases, since the same tables are used multiple times.

Self-Joins

Not only can you include the same table more than once in the same query, but you can actually join a table to itself. This might seem like a strange thing to do at first, but there are valid reasons for doing so. Some tables include a *self-referencing foreign key*, which

means that it includes a column that points to the primary key within the same table. While the sample database doesn't include such a relationship, let's imagine that the `film` table includes the column `prequel_film_id`, which points to the film's parent (e.g., the film *Fiddler Lost II* would use this column to point to the parent film *Fiddler Lost*). Here's what the table would look like if we were to add this additional column:

```
mysql> desc film;
+----------------------+----------------------+------+-----+------------------+
| Field                | Type                 | Null | Key | Default          |
+----------------------+----------------------+------+-----+------------------+
| film_id              | smallint(5) unsigned | NO   | PRI | NULL             |
| title                | varchar(255)         | NO   | MUL | NULL             |
| description          | text                 | YES  |     | NULL             |
| release_year         | year(4)              | YES  |     | NULL             |
| language_id          | tinyint(3) unsigned  | NO   | MUL | NULL             |
| original_language_id | tinyint(3) unsigned  | YES  | MUL | NULL             |
| rental_duration      | tinyint(3) unsigned  | NO   |     | 3                |
| rental_rate          | decimal(4,2)         | NO   |     | 4.99             |
| length               | smallint(5) unsigned | YES  |     | NULL             |
| replacement_cost     | decimal(5,2)         | NO   |     | 19.99            |
| rating               | enum('G','PG','PG-13',
                       'R','NC-17')         | YES  |     | G                |
| special_features     | set('Trailers',...,
                       'Behind the Scenes')| YES  |     | NULL             |
| last_update          | timestamp            | NO   |     | CURRENT_
                                             TIMESTAMP        |
| prequel_film_id      | smallint(5) unsigned | YES  | MUL | NULL             |
+----------------------+----------------------+------+-----+------------------+
```

Using a *self-join*, you can write a query that lists every film that has a prequel, along with the prequel's title:

```
mysql> SELECT f.title, f_prnt.title prequel
    -> FROM film f
    ->   INNER JOIN film f_prnt
    ->   ON f_prnt.film_id = f.prequel_film_id
    -> WHERE f.prequel_film_id IS NOT NULL;
+-----------------+--------------+
| title           | prequel      |
+-----------------+--------------+
| FIDDLER LOST II | FIDDLER LOST |
+-----------------+--------------+
1 row in set (0.00 sec)
```

This query joins the `film` table to itself using the `prequel_film_id` foreign key, and the table aliases `f` and `f_prnt` are assigned in order to make it clear which table is used for which purpose.

Test Your Knowledge

The following exercises are designed to test your understanding of inner joins. Please see Appendix B for the solutions to these exercises.

Exercise 5-1

Fill in the blanks (denoted by `< # >`) for the following query to obtain the results that follow:

```
mysql> SELECT c.first_name, c.last_name, a.address, ct.city
    -> FROM customer c
    ->   INNER JOIN address <1>
    ->   ON c.address_id = a.address_id
    ->   INNER JOIN city ct
    ->   ON a.city_id = <2>
    -> WHERE a.district = 'California';
+------------+-----------+-----------------------+----------------+
| first_name | last_name | address               | city           |
+------------+-----------+-----------------------+----------------+
| PATRICIA   | JOHNSON   | 1121 Loja Avenue      | San Bernardino |
| BETTY      | WHITE     | 770 Bydgoszcz Avenue  | Citrus Heights |
| ALICE      | STEWART   | 1135 Izumisano Parkway | Fontana       |
| ROSA       | REYNOLDS  | 793 Cam Ranh Avenue   | Lancaster      |
| RENEE      | LANE      | 533 al-Ayn Boulevard  | Compton        |
| KRISTIN    | JOHNSTON  | 226 Brest Manor       | Sunnyvale      |
| CASSANDRA  | WALTERS   | 920 Kumbakonam Loop   | Salinas        |
| JACOB      | LANCE     | 1866 al-Qatif Avenue  | El Monte       |
| RENE       | MCALISTER | 1895 Zhezqazghan Drive | Garden Grove  |
+------------+-----------+-----------------------+----------------+
9 rows in set (0.00 sec)
```

Exercise 5-2

Write a query that returns the title of every film in which an actor with the first name JOHN appeared.

Exercise 5-3

Construct a query that returns all addresses that are in the same city. You will need to join the address table to itself, and each row should include two different addresses.

Previous chapter
< 4. Filtering

Next chapter
6. Working with Sets >